

LOOP CONTROL FLOW

Mastering Break & Continue Statements

CodeHive Academy

Module 10: Precision Iteration

1. The 'break' Statement

The break statement is used to "jump out" of a loop entirely, regardless of the loop's original condition.

- **Function:** It immediately terminates the loop and moves the program to the next line of code outside that loop.
- **Common Use:** Often used to stop a search once a specific value is found to save processing time.

```
for (int i = 0; i < 10; i++) { if (i == 4) { break; // Stops the loop  
completely when i reaches 4 } cout << i << "\n"; }
```

```
0  
1  
2  
3
```

2. Real-Life Logic: The Search Engine

Imagine you are building a search feature for the **CodeHive platform**. Once you find the specific user or problem you are looking for, there is no reason to continue scanning thousands of other entries.

```
int luckyNumber = 7; for (int i = 1; i ≤ 100; i++) { if (i == luckyNumber)
{ cout << "Found it at position: " << i << "\n"; break; // Target found!
Stop and save power. } cout << "Checking position " << i << "... \n"; }
```

CodeHive Engineering Tip: Without `break`, the computer would waste time checking the other 93 positions even though the task is finished. Efficiency is key!

3. The 'continue' Statement

While `break` exits the loop, `continue` simply "skips" the current iteration and jumps to the next turn of the loop.

- **Function:** It stops the current cycle and jumps immediately to the increment/decrement statement.
- **Common Use:** Used when you want to ignore certain values (like skipping even numbers or forbidden items) but still finish the rest of the loop.

```
for (int i = 0; i < 5; i++) { if (i == 2) { continue; // Skips the rest of  
the code ONLY for i = 2 } cout << i << "\n"; }
```

```
0  
1  
3  
4
```

4. Real-Life Logic: The Filter System

Imagine building a "Leaderboard" on your platform, but you want to skip users who have a score of zero to keep the list clean. You don't want to stop the whole list; you just want to skip the "empty" entries.

The "Unlucky 13" Example:

```
for (int floor = 1; floor ≤ 15; floor++) { if (floor = 13) { continue; //  
Skip floor 13 and jump straight to 14. } cout << "Welcome to Floor " <<  
floor << "\n"; }
```

Note: The loop continues running until it reaches its final limit; only the specific iteration for 13 is ignored.

5. Comparison & Key Differences

Understanding when to use each statement is vital for clean control flow architecture.

Feature	Break	Continue
Main Action	Terminates the whole loop.	Skips just one turn.
Next Step	Line after the loop block.	Next iteration start.
Typical Use	Searching for unique items.	Filtering/Cleaning data.

6. Practice Summary

Mastering these two keywords allows you to control the "rhythm" of your code. You are no longer a passenger in your loop; you are the driver.

✂ Skill Challenge

Write a program that loops through numbers 1 to 20. Use `continue` to skip all even numbers, but use `break` to stop the loop entirely if the number reaches 15.

Code. Learn. Repeat.

© 2026 CodeHive Academy. Module: Controlling Iteration Flow.